AutoAdvisor AI

Full Project Roadmap and Implementation Workflow

An AutoTrader-inspired AI car-buying assistant with car recommendations, RAG, ML price estimation, review intelligence, dealership connection, and vehicle image analysis.

Prepared as starter context for a new ChatGPT Project and as an implementation guide for a 2-week solo AI engineering build.

1. Project Context Prompt for a New ChatGPT Project

Use this section as the first message or project context when starting a new ChatGPT Project.

> I am building AutoAdvisor AI, a solo 2-week AI engineering project. It is an AutoTrader-inspired car-buying assistant that helps users find, compare, and evaluate cars based on budget, lifestyle, preferences, reviews, uploaded car images, image quality checks, safety checks, and available dealer/demo inventory. The project should avoid unauthorized live scraping of restricted marketplaces. It can use public datasets, official APIs, approved listing APIs, seeded inventory, curated review documents, and permitted scraping only when allowed. The app should include a FastAPI backend, Streamlit frontend, SQLite database, ChromaDB vector store, RAG over car-buying guides/reviews, ML models for intent classification, used-car price estimation, review sentiment/topic analysis, and pretrained DL/computer-vision models for image analysis and NSFW checks. The system should produce explainable recommendations and optionally create a dealership inquiry draft or lead record after user confirmation.

2. One-Line Description

AutoAdvisor AI is an intelligent car-buying assistant that helps users find, compare, and evaluate cars based on budget, lifestyle, preferences, reviews, image analysis, and available dealer inventory.

3. Problem Statement

Buying a used car is confusing because users often do not know which models fit their budget, lifestyle, maintenance tolerance, fuel needs, or comfort expectations. Marketplaces provide filters, but users still need to know what to search for, how to compare cars, and whether a specific listing looks reasonable.

AutoAdvisor AI solves this by turning natural-language car needs and optional uploaded car photos into structured preferences, filtered results, price checks, review summaries, visual inspection signals, and dealer inquiry drafts.

4. Target Users

- First-time buyer: wants a cheap, reliable, low-maintenance car.
- City driver: wants compact size, easy parking, and good fuel economy.
- Family buyer: wants space, practicality, safety, and reasonable running costs.
- Luxury buyer: wants a premium car but needs maintenance and ownership-cost warnings.
- Dealer/admin demo user: uploads a listing photo and expects metadata assistance, quality checks, and safety screening.

5. Final Scope

In Scope

- Chatbot that accepts natural-language car needs.
- Structured preference extraction: budget, use case, body type, brand, fuel, mileage, year, location.
- Used-car search/filtering over public, seeded, or approved datasets.
- Recommendation of 3 to 5 cars with reasons and tradeoffs.
- Car comparison between two or more models.
- RAG over car-buying guides, vehicle knowledge, maintenance notes, and review summaries.
- User ratings and minimal feedback summaries when review data is available.
- Dealer connection workflow: inquiry draft or demo lead record after user confirmation.

- Marketplace availability based on seeded demo inventory, uploaded dealership data, or approved APIs.
- ML models for intent classification, price estimation, and review sentiment/topic analysis.
- Pretrained DL/computer vision for vehicle presence/type/color/make-model assistance, NSFW checks, and image quality checks.
- Evaluation sets for classifier, extraction, retrieval, price model, review sentiment, image checks, and end-to-end recommendations.

Out of Scope

- Unauthorized live scraping of AutoTrader, Dubizzle, Cars.com, CarGurus, or other restricted marketplaces.
- Guaranteed real-time marketplace availability without an approved source.
- Payment, financing approval, insurance quotes, legal advice, or financial advice.
- Automatic message sending to dealers without user confirmation.
- Training a large deep-learning car recognition model from scratch.
- Enterprise multi-tenancy, Kubernetes, event buses, microservices, and advanced DevOps.

6. Main User Workflows

Workflow A - Car Recommendation

```
User message
  -> Intent classifier
  -> Preference extractor
  -> Structured car search
  -> Price estimator + fit scorer
  -> RAG retrieval over guides/reviews
  -> Recommendation generator
  -> Response with cars, reasons, tradeoffs, sources, and optional dealer action
```

Workflow B - Car Image Analysis

```
Uploaded image
  -> NSFW safety checker
  -> Image quality checker
  -> Vehicle detector
  -> Attribute extraction: type, color, optional make/model
  -> Confidence validation
  -> Use extracted attributes in search/recommendation OR ask user to confirm
```

Workflow C - Dealer Connection

```
User selects a car
  -> Dealer inquiry intent detected
  -> Match dealer/demo inventory by make, model, location, budget
  -> Generate inquiry draft
  -> Ask for confirmation
  -> Store demo lead or show copy/paste message
```

7. Proposed Architecture

```
Streamlit Frontend
  - Chat assistant page
  - Car search page
  - Car comparison page
  - Image upload page
  - Evaluation dashboard

FastAPI Backend
  - /chat
  - /recommend
  - /compare
  - /image/analyze
  - /dealer/inquiry
  - /search/cars
  - /eval/results

AI Services
  - Intent classifier (ML)
  - Preference extractor (LLM + schema)
  - Price estimator (ML regression)
  - Review sentiment/topic classifier (ML)
  - RAG retriever (embeddings + ChromaDB)
  - Recommendation generator (LLM + rules)
  - Vehicle image analyzer (pretrained DL/CV)
  - NSFW image checker (pretrained DL)
  - Quality checker (OpenCV)

Storage
  - SQLite: cars, reviews, dealers, leads, eval results
  - ChromaDB: guide/review document embeddings
  - Local file storage: uploaded demo images, cleaned datasets, model artifacts
```

8. Data Strategy

Data Source	Purpose	Recommended Use
Kaggle / public used-car listings	Inventory-like data: price, year, mileage, make, model, condition	Primary demo inventory and model training data
NHTSA vPIC API	Make/model/VIN metadata and validation	Optional enrichment and validation
FuelEconomy.gov / EPA data	MPG, vehicle class, fuel type, annual fuel cost	Fuel-economy enrichment and city-car scoring
Curated car guides/reviews	RAG knowledge base	Use public pages or manually saved allowed content
MarketCheck or approved listing API	Real or semi-real listings/dealer data	Optional if access is available
Seeded dealer inventory	Demo availability and dealer connection	Reliable fallback for presentation

> The project should not depend on restricted live scraping. If scraping is included, it should be limited to permitted public pages and used as an ingestion demonstration, not the core data source.

9. AI, ML, and DL Components

Component	Model Type	Purpose	Difficulty
Intent classifier	TF-IDF + Logistic Regression	Route messages to recommendation, comparison, dealer inquiry, image analysis, etc.	Easy
Preference extraction	LLM + Pydantic schema	Extract budget, car type, use case, brand, mileage, year, location	Medium
Used-car price estimator	Random Forest / Gradient Boosting Regressor	Estimate fair price and label listings as good/fair/overpriced	Medium
Recommendation fit scorer	Rule-based + optional ML	Rank cars against user needs	Medium
Review sentiment/topic classifier	Logistic Regression or pretrained sentiment model	Summarize positives/complaints by model	Medium
RAG retriever	Embeddings + ChromaDB	Ground advice in car guides/reviews	Medium
Vehicle image analyzer	Pretrained DL/CV	Detect car presence, body type, color, optional make/model	Medium/Hard
NSFW image checker	Pretrained image safety classifier	Reject unsafe image uploads	Easy/Medium
Image quality checker	OpenCV	Detect blur, darkness, low resolution, bad framing	Easy

10. Vehicle Image Intelligence

This module handles uploaded car photos. It is designed to assist the user or dealer, not to guarantee perfect vehicle identification. Low-confidence predictions must be shown as suggestions and should be confirmed by the user.

10.1 Image Safety Gate - NSFW Checker

- Runs before any image is stored, displayed, or analyzed further.
- Rejects or quarantines images classified as NSFW above a threshold.
- Stores only safe images for demo analysis.
- Returns a safe, simple refusal message for inappropriate uploads.
- Uses a pretrained classifier such as a lightweight Hugging Face NSFW detector.

10.2 Image Quality Checker

- Resolution check: reject or warn if the image is too small.
- Blur check: use Laplacian variance or similar OpenCV method.
- Brightness check: warn if too dark or overexposed.
- Vehicle visibility check: require a detected car with reasonable confidence.
- Framing check: warn if the car occupies too little of the image.

10.3 Vehicle Attribute Extraction

- Vehicle presence: car, truck, bus, motorcycle detection if model supports it.
- Body type: sedan, SUV, hatchback, coupe, pickup, van where feasible.
- Color: estimate dominant vehicle color after vehicle crop/detection.
- Make/model: optional, only when pretrained model/API gives high confidence.
- Confidence handling: uncertain predictions are treated as suggestions, not facts.

10.4 Suggested Image Output

```
{
  "safe": true,
  "quality": {
    "status": "acceptable",
    "blur_score": 142.7,
    "brightness": "normal",
    "resolution": "1280x720"
  },
  "vehicle": {
    "detected": true,
    "type": "sedan",
    "type_confidence": 0.82,
    "color": "white",
    "make_model_guess": "Toyota Corolla",
    "make_model_confidence": 0.61,
    "requires_user_confirmation": true
  }
}
```

Important implementation rule: exact make/model recognition from images is a stretch feature. The minimum viable image module should support safety, quality, car presence, body type, and color.

11. Backend API Design

Endpoint	Purpose
GET /health	Basic backend health check
POST /chat	Main assistant endpoint
POST /recommend	Recommendation workflow from structured preferences
POST /compare	Compare two or more cars
POST /classify	Debug/evaluate intent classifier
POST /extract	Debug/evaluate preference extraction
GET /search/cars	Search/filter car inventory
POST /image/analyze	Run NSFW, quality, and vehicle attribute checks
POST /dealer/inquiry	Generate/store dealership inquiry draft
GET /eval/results	Return latest evaluation metrics

12. Database Design

Table	Purpose	Key Fields
cars	Used-car inventory/demo listings	make, model, year, price, mileage, fuel, transmission, body_type, location, platform_source, availability
car_reviews	Ratings and minimal feedback	make, model, year, rating, review_text, sentiment, topics, source
dealerships	Dealer contact/demo matching	name, location, phone, email, website, supported_brands
dealer_leads	Stored demo inquiry requests	selected_car, budget, location, contact_preference, message_draft, status
documents	RAG source docs	title, url, category, raw_text
document_chunks	RAG chunk metadata	document_id, chunk_index, text, category, source_url
image_analysis	Image analysis results	image_id, nsfw_score, quality_status, type, color, make_model_guess
eval_results	Evaluation metrics	eval_name, metric_name, metric_value, created_at

13. Frontend Pages

- Chat Assistant: natural-language recommendation and Q&A.;
- Car Search: manual filters for budget, make, model, year, mileage, fuel, and location.
- Compare Cars: side-by-side comparison using structured data and RAG explanations.
- Image Analysis: upload car image, show NSFW result, quality result, type/color/make-model suggestion.
- Dealer Connection: show matching dealers/demo inventory and inquiry draft.
- Evaluation Dashboard: show ML, RAG, image, and end-to-end metrics.

14. Implementation Roadmap

Phase 0 - Setup Decisions

- Confirm stack: FastAPI, Streamlit, SQLite, ChromaDB, scikit-learn, OpenCV, Hugging Face/Transformers or selected API models.
- Choose whether image models run locally or through API. Local models are more educational but may be heavier.
- Define MVP vs stretch features. MVP should not depend on exact make/model image recognition or live marketplace APIs.

Phase 1 - Core Data Foundation

- Download or prepare used-car dataset.
- Normalize columns: make, model, year, price, mileage, fuel, transmission, body type, condition, location.
- Seed demo dealerships and demo availability.
- Prepare review/rating sample if available.
- Create SQLite schema and ingestion scripts.

Phase 2 - Backend and Frontend Skeleton

- Create FastAPI app with health endpoint.
- Create Streamlit app with placeholder pages.
- Connect Streamlit to FastAPI.
- Add .env.example and basic config handling.
- Add README startup instructions.

Phase 3 - Classical ML Models

- Train intent classifier on manually labeled examples.
- Train used-car price estimator from structured dataset.
- Train or implement review sentiment/topic classifier.
- Save model artifacts with joblib.
- Expose classifier and price-check endpoints.

Phase 4 - RAG Knowledge Base

- Collect allowed car-buying guides and review summaries.
- Chunk documents with metadata: title, category, model, source URL.
- Generate embeddings and store in ChromaDB.
- Implement retrieval service.
- Create a small retrieval golden set.

Phase 5 - Recommendation Workflow

- Implement preference extraction schema.
- Search/filter inventory based on extracted preferences.

- Run price estimator on candidate cars.
- Rank results with fit score.
- Retrieve RAG guidance and review insights.
- Generate final recommendation with tradeoffs and sources.

Phase 6 - Image Intelligence

- Implement upload endpoint and file validation.
- Run NSFW check before storing or analyzing image.
- Implement OpenCV quality checks: resolution, blur, brightness.
- Run vehicle detector/body-type classifier if selected.
- Implement color extraction from detected/cropped vehicle region.
- Add optional make/model classifier only if feasible.
- Return confidence scores and ask for confirmation when uncertain.

Phase 7 - Dealer Connection

- Create dealership table and seed data.
- Match dealer by location, brand, and availability.
- Generate inquiry draft.
- Require user confirmation before storing lead.
- Do not send messages automatically in the MVP.

Phase 8 - Evaluation and Polish

- Run evals for intent classifier, price estimator, extraction, RAG, sentiment, image safety, image quality, and recommendations.
- Add smoke tests for chat, image upload, and dealer inquiry.
- Finalize README, DESIGN.md, DATA.md, EVALS.md, RUNBOOK.md.
- Prepare demo script with city car, luxury car, family SUV, comparison, image upload, and dealer inquiry.

15. Two-Week Schedule

Day	Goal	Main Deliverables
1	Skeleton and design	Repo, FastAPI, Streamlit, SQLite setup, README outline
2	Data ingestion	Cars table, dealers table, reviews table, search endpoint
3	Intent classifier	Training data, TF-IDF model, /classify endpoint, first metrics
4	Price estimator	Regression model, price label logic, /price-check behavior
5	RAG setup	Documents, chunks, embeddings, retrieval endpoint, retrieval eval set
6	Preference extraction	Schema, extraction prompt/service, validation, /extract endpoint
7	Recommendation engine	Search + fit score + price model + RAG recommendation
8	Image safety and quality	NSFW checker, blur/brightness/resolution checks, image endpoint
9	Vehicle image attributes	Vehicle detection/type/color extraction, confidence handling
10	Dealer workflow	Dealer matching, inquiry draft, lead storage after confirmation
11	Frontend polish	Chat, search, comparison, image, dealer, eval pages
12	Evaluations	Classifier, RAG, price, sentiment, image, smoke tests
13	Docs	DESIGN.md, DATA.md, EVALS.md, RUNBOOK.md
14	Demo readiness	Final checks, demo script, tagged release

16. MVP vs Stretch Features

Feature	MVP	Stretch
Car recommendation	3 to 5 cars from local/public dataset	Live approved API inventory
Ratings/feedback	Small review sample and sentiment summary	Large review dataset with topic modeling
Dealer connection	Seeded dealers + inquiry draft	Approved dealer API or email workflow
Price estimator	Random Forest/Gradient Boosting regression	Model explanations with SHAP
Image safety	Pretrained NSFW classifier	Moderation dashboard
Image quality	OpenCV blur/brightness/resolution	Learned quality classifier
Vehicle type/color	Pretrained detector + simple color extraction	Fine-tuned body-type classifier
Make/model from image	Optional high-confidence suggestion	Dedicated pretrained/fine-tuned model
Scraping	Permitted pages only, optional	Approved API ingestion pipeline

17. Evaluation Plan

Evaluation	Dataset	Metric	Target
Intent classification	50-100 labeled messages	Accuracy, macro-F1	>= 80% accuracy
Preference extraction	25-40 scenarios	Valid JSON, field accuracy	>= 95% valid JSON, >= 80% key fields
Price estimator	Held-out car listings	MAE/RMSE, price band accuracy	Report baseline vs chosen model
RAG retrieval	15-25 questions	hit@3, hit@5, MRR	hit@3 >= 70%, hit@5 >= 80%
Review sentiment/topic	Small labeled review set	Accuracy/F1	Report confusion matrix
NSFW checker	Safe/unsafe sample set	False positive/negative review	Unsafe images blocked
Image quality	Good/blurry/dark/low-res set	Pass/fail accuracy	Most poor images flagged
Vehicle attributes	Small labeled image set	Type/color accuracy	Report confidence and failures
End-to-end recommendation	10 scenarios	Manual pass/fail rubric	8/10 pass

18. Safety, Compliance, and Reliability

- No unauthorized scraping of restricted marketplaces.
- Clearly label seeded/demo inventory as demo inventory.
- Do not claim real-time availability unless using an approved live source.
- Do not provide financing, insurance, or legal advice.
- Do not send dealer inquiries without explicit user confirmation.
- Redact phone numbers/emails from logs where not needed.
- Run NSFW checks before saving or processing uploaded images.
- Show confidence scores for image predictions and require confirmation for uncertain make/model guesses.
- Keep API keys in .env and provide .env.example only.

19. Recommended Repository Structure

```
autoadvisor-ai/
  README.md
  DESIGN.md
  DATA.md
  EVALS.md
  RUNBOOK.md
  .env.example
  requirements.txt

  backend/
    app/
      main.py
      config.py
      api/
        chat_routes.py
        car_routes.py
        image_routes.py
        dealer_routes.py
        eval_routes.py
      services/
        chat_service.py
        recommendation_service.py
        preference_extraction_service.py
        retrieval_service.py
        price_service.py
        review_service.py
        dealer_service.py
        image_safety_service.py
        image_quality_service.py
        vehicle_vision_service.py
      models/
        schemas.py
        intent_classifier.joblib
        price_estimator.joblib
        review_classifier.joblib
      db/
        database.py
        init_db.py
      prompts/
        preference_extraction.md
        recommendation.md
        comparison.md

  frontend/
    streamlit_app.py
    pages/
      1_Chat_Assistant.py
      2_Car_Search.py
      3_Compare_Cars.py
      4_Image_Analysis.py
      5_Dealer_Inquiry.py
      6_Evaluation_Dashboard.py

  data/
    raw/
    processed/
    docs/
    vectorstore/
    images_demo/
    eval/

  scripts/
    ingest_cars.py
    ingest_reviews.py
    ingest_docs.py
    seed_dealers.py
    train_intent_classifier.py
    train_price_estimator.py
    train_review_classifier.py
    build_vector_index.py
    run_all_evals.py
```

```
tests/
  test_chat_flow.py
  test_recommendation.py
  test_price_model.py
  test_retrieval.py
  test_image_safety.py
  test_image_quality.py
  test_dealer_inquiry.py
```

20. Demo Script

Demo 1 - City Car Recommendation

Input: I need a reliable city car under $10,000 with low fuel consumption.

- Show extracted budget/use case.
- Show filtered cars.
- Show price estimate and fit score.
- Show RAG explanation and sources.

Demo 2 - Luxury Car With Maintenance Concern

Input: I want a fancy luxury car but I do not want expensive maintenance.

- Show luxury intent.
- Retrieve maintenance tradeoff guidance.
- Recommend safer options or warn about older luxury cars.

Demo 3 - Family SUV

Input: I need an SUV for my family, budget around $18,000, good fuel economy.

- Show SUV/family/fuel priorities.
- Show fit-ranked SUV options.
- Show tradeoffs.

Demo 4 - Car Comparison

Input: Compare Toyota Corolla and Honda Civic for a first car.

- Show comparison table.
- Show recommendation depending on priorities.

Demo 5 - Image Upload

- Upload safe, clear car image.
- Show NSFW pass.
- Show quality score.
- Show detected vehicle type/color and optional make/model suggestion.
- Use detected attributes in a search or recommendation.

Demo 6 - Dealer Inquiry

Input: I like the Toyota Corolla option. Can you connect me with a dealership?

- Show matching seeded dealer/demo inventory.
- Generate inquiry draft.
- Ask for user confirmation before storing/sending anything.

21. Final Deliverables

- GitHub repository with clean project structure.
- Working FastAPI backend and Streamlit frontend.
- SQLite database with car, review, dealer, and lead tables.
- ChromaDB vector store for RAG.
- ML artifacts: intent classifier, price estimator, review classifier.
- Image pipeline: NSFW checker, image quality checker, vehicle attribute extraction.
- Dealer inquiry workflow.
- Evaluation scripts and metrics.
- README.md, DESIGN.md, DATA.md, EVALS.md, RUNBOOK.md.
- Demo script and sample inputs/images.

22. Key Risks and Mitigations

Risk	Mitigation
Marketplace scraping is restricted or unstable	Use public datasets, approved APIs, seeded inventory, and permitted pages only
Image make/model detection is inaccurate	Treat as optional suggestion with confidence and user confirmation
Project becomes too large	Prioritize MVP: recommendation, price model, RAG, NSFW, quality, type/color
Live API access fails during demo	Use local cached/seeded data fallback
LLM outputs unsupported claims	Ground responses in retrieved docs and structured data
User uploads inappropriate images	Run NSFW gate before storage or analysis

23. Suggested Source References

These references are useful as candidate data/model sources. Access, licensing, rate limits, and terms should be checked before implementation.

- NHTSA vPIC API - vehicle make/model and VIN decoding: https://vpic.nhtsa.dot.gov/api/
- NHTSA datasets and APIs overview: https://www.nhtsa.gov/nhtsa-datasets-and-apis
- EPA/FuelEconomy.gov vehicle data: https://www.epa.gov/compliance-and-fuel-economy-data/data-cars-used-testing-fuel-economy
- MarketCheck Cars API - approved listing/dealer data option: https://www.marketcheck.com/apis/cars/
- Hugging Face NSFW model example: https://huggingface.co/Marqo/nsfw-image-detection-384

- Kaggle public used-car datasets, for example Craigslist Cars and Trucks datasets.

24. Recommended Build Priority

If time becomes tight, build in this order:

- 1. Core car search and recommendation over public/seeded inventory.
- 2. Intent classifier and preference extraction.
- 3. Used-car price estimator.
- 4. RAG over car guides and review summaries.
- 5. Dealer inquiry draft over seeded dealerships.
- 6. NSFW image checker and image quality checker.
- 7. Vehicle type and color extraction.
- 8. Optional make/model image recognition.
- 9. Optional approved live API integration.

> This order ensures the project remains demo-ready even if the computer-vision or live-data components need to be simplified.